

AIR - Agentic SRE Implementation Guide

Planner, Retry, Evidence Agents, Evidence Intelligence and the path to RCA

Purpose
A section-based conversion of the implementation discussion into a single engineering reference for building AIR's Agentic Incident Response workflow.

Coverage	Investigation Planner, planner retry, evidence agent architecture, LogsAgent, MetricsAgent, DeploymentAgent, Recent Incident Agent, evidence aggregation and InvestigationKnowledge
Architecture style	LangGraph orchestration + thin nodes + service-layer agents + tool adapters + structured outputs
Primary design principle	Evidence -> Knowledge -> Reasoning

Contents

Contents	2
1. Investigation Planner - Core Design	4
1.1 Plan-and-Execute mental model	4
1.2 Planner structured output	4
1.3 State versus planner output	4
1.4 Thin-node architecture	5
2. Planner Retry and Re-Planning	5
2.1 Planner-time retry loop	5
2.2 Retry state	5
2.3 Deterministic readiness gate	6
2.4 Conditional routing	6
2.5 Technical retry vs reasoning retry	6
2.6 Later re-planning loop	6
3. Evidence Collection Architecture	6
3.1 Shared Evidence contract	7
3.2 BaseEvidenceAgent	7
4. LogsAgent - ELF Evidence Collection	7
4.1 Flow	8
4.2 Request model	8
4.3 Normalized log model	8
4.4 Pattern extraction	8
4.5 LLM use inside LogsAgent	9
4.6 Retry behavior	9
4.7 Example evidence	9
5. MetricsAgent - Runtime Health Evidence	9
5.1 Flow	10
5.2 Request model	10
5.3 Normalized metric model	10
5.4 Deterministic analysis	10
5.5 Baseline comparison	10
5.6 Example evidence	11
6. DeploymentAgent - GitHub and Change Evidence	11
6.1 Flow	11
6.2 Request model	11
6.3 Normalized event	12
6.4 Deterministic analysis	12
6.5 Example evidence	12
7. RecentIncidentAgent - Historical Context	12
7.1 Architecture	13

7.2 Request model	13
7.3 Deterministic similarity first	13
7.4 Enterprise semantic search	13
7.5 Example evidence	14
8. Evidence Aggregation and InvestigationKnowledge	14
8.1 Evidence Aggregator responsibilities	14
8.2 AggregatedEvidence model	14
8.3 Evidence Intelligence Engine	14
8.4 InvestigationKnowledge model	15
8.5 Example knowledge object	15
9. Handoff to Root Cause Assessment	15
9.1 RCA output	15
9.2 Full end-to-end flow	16
10. Engineering Rules to Preserve	16
10.1 Suggested next implementation milestone	16
Appendix A. Suggested Repository Layout	17
Appendix B. Testing Checklist	17

1. Investigation Planner - Core Design

The Planner is the first AI reasoning component in AIR. Its output is an executable investigation strategy, not a root-cause conclusion. It classifies the incident, selects specialist evidence agents, specifies required evidence and decides which investigations can run in parallel.

Hard boundary

The Planner decides what to investigate. Evidence agents determine what was observed. The RCA Agent determines what the evidence most likely means.

1.1 Plan-and-Execute mental model

```

Incident Context
  |
  v
Context Builder
  |
  v
Planner Agent
  |
  +--> classify incident
  +--> select specialist agents
  +--> request evidence
  +--> identify parallel work
  |
  v
Investigation Plan
  |
  v
Feature 3 Evidence Executors

```

1.2 Planner structured output

```

class PlannerOutput(BaseModel):
    plannerVersion: str
    status: Literal["READY", "NEEDS_MORE_CONTEXT"]
    confidence: float = Field(ge=0.0, le=1.0)
    investigationType: str | None = None
    priority: str | None = None
    parallelAgents: list[str] = []
    requiredEvidence: list[str] = []
    hypotheses: list[str] = []
    missingContext: list[str] = []
    contextRequests: list[ContextRequest] = []
    reasoning: str

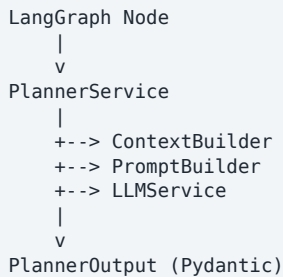
```

The hypotheses field contains investigation leads only. A hypothesis such as "Deployment Regression" means "investigate whether a deployment regression occurred"; it is not an RCA conclusion.

1.3 State versus planner output

Concept	Purpose
LangGraph State	Tracks workflow position, iteration counts, status and shared runtime data.
PlannerOutput	Captures the Planner decision for what Feature 3 should execute.

1.4 Thin-node architecture

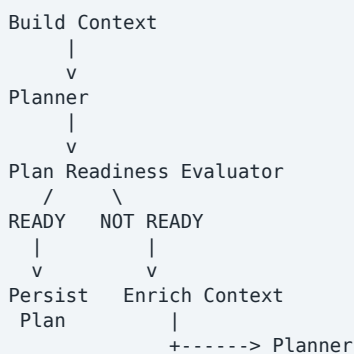


LangGraph orchestrates; services implement. Prompt text, API calls, SQL and business rules should not live inside the node.

2. Planner Retry and Re-Planning

When the Planner does not have enough information or does not meet the readiness threshold, AIR should enter a bounded planning loop. The retry must enrich or change the context; repeatedly asking the model the same question with the same input is not a meaningful retry.

2.1 Planner-time retry loop



2.2 Retry state

```

class InvestigationState(TypedDict):
    investigation_id: str
    incident: dict
    planner_context: dict
    planner_output: dict | None
    planner_iteration: int
    max_planner_iterations: int
    planner_retry_reason: str | None
    status: str
    messages: list[str]
  
```

2.3 Deterministic readiness gate

```
CONFIDENCE_THRESHOLD = 0.75

def is_plan_ready(plan: PlannerOutput) -> bool:
    if plan.status != "READY":
        return False
    if plan.confidence < CONFIDENCE_THRESHOLD:
        return False
    if not plan.investigationType:
        return False
    if not plan.parallelAgents:
        return False
    if not plan.requiredEvidence:
        return False
    return True
```

Do not trust LLM confidence alone

Plan readiness should combine schema validity, mandatory context, valid agent names, evidence coverage and model confidence. In production, deterministic checks should dominate the gate.

2.4 Conditional routing

```
def route_after_planner(state: InvestigationState) -> str:
    plan = PlannerOutput.model_validate(state["planner_output"])

    if is_plan_ready(plan):
        return "ready"

    if state["planner_iteration"] >= state["max_planner_iterations"]:
        return "retry_exhausted"

    return "retry"
```

A reasonable initial configuration is three planner iterations. If the limit is exhausted, route to `PLANNER_REVIEW_REQUIRED` rather than silently continuing with a weak plan.

2.5 Technical retry vs reasoning retry

Retry Type	Owned By	Examples
Technical retry	LLMService / Tool Adapter	Timeout, 429, transient 5xx, network failure
Reasoning retry	LangGraph workflow	Low confidence, missing context, incomplete investigation coverage

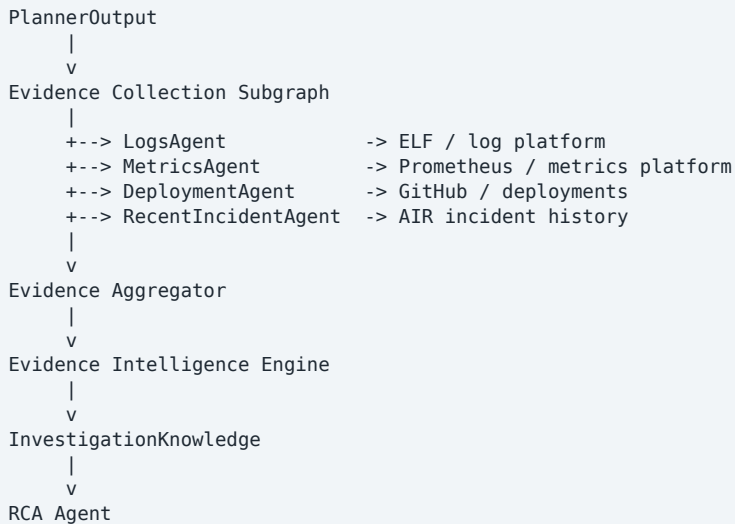
2.6 Later re-planning loop

After evidence collection, low RCA confidence is a different problem. That should trigger a Re-Planner that receives the existing plan, collected evidence, unresolved questions and RCA confidence, then returns a plan delta for additional evidence rather than rebuilding the investigation from scratch.

```
Plan -> Execute -> Evaluate -> Re-plan
      ^           |
      |-----|
```

3. Evidence Collection Architecture

Once the Planner is ready, Feature 3 executes independent evidence specialists. Each specialist collects facts from one domain and returns a common Evidence contract. Agents never call one another directly; LangGraph coordinates them through shared state.



3.1 Shared Evidence contract

```

class Evidence(BaseModel):
    investigation_id: UUID
    agent_name: str
    evidence_type: EvidenceType
    source_system: str
    title: str
    summary: str
    confidence: float = Field(ge=0.0, le=1.0)
    findings: dict[str, Any]
    references: list[EvidenceReference] = []
    observed_from: datetime | None = None
    observed_to: datetime | None = None
    collected_at: datetime

```

3.2 BaseEvidenceAgent

```

class BaseEvidenceAgent(ABC):
    @abstractmethod
    async def collect(self, request): ...

    @abstractmethod
    def normalize(self, raw): ...

    @abstractmethod
    def summarize(self, normalized, request) -> list[Evidence]: ...

    async def execute(self, request):
        raw = await self.collect(request)
        normalized = self.normalize(raw)
        return self.summarize(normalized, request)

```

Agent boundary

Evidence agents return observable facts. They do not generate root cause and do not recommend remediation.

4. LogsAgent - ELF Evidence Collection

LogsAgent owns application and exception logs, error patterns, stack traces and log frequency. It should collect and normalize facts from ELF while keeping vendor-specific query syntax behind an adapter.

4.1 Flow

```

LogEvidenceRequest
  |
  v
LogsQueryBuilder
  |
  v
LogTool -> ELF Adapter
  |
  v
Raw Log Results
  |
  v
LogNormalizer
  |
  v
LogPatternExtractor
  |
  v
EvidenceBuilder
  |
  v
Evidence[]

```

4.2 Request model

```

class LogEvidenceRequest(BaseModel):
    investigation_id: UUID
    service_name: str
    environment: str
    start_time: datetime
    end_time: datetime
    required_evidence: list[str] = []
    hypothesis_context: list[str] = []
    namespace: str | None = None
    cluster: str | None = None
    pod: str | None = None
    max_results: int = Field(default=5000, ge=1, le=50000)

```

4.3 Normalized log model

```

class NormalizedLogEvent(BaseModel):
    timestamp: datetime
    service: str
    environment: str
    level: str
    message: str
    exception_type: str | None = None
    exception_message: str | None = None
    class_name: str | None = None
    method_name: str | None = None
    trace_id: str | None = None
    span_id: str | None = None
    host: str | None = None
    pod: str | None = None
    namespace: str | None = None
    http_status: int | None = None

```

4.4 Pattern extraction

The deterministic extractor should calculate exception counts, first/last seen times, affected classes, affected pods, HTTP status counts and event frequency. Do not ask an LLM to count occurrences that software can calculate exactly.

```
findings = {
  "totalEvents": len(events),
  "errorCount": len(errors),
  "warningCount": len(warnings),
  "exceptions": exception_counts,
  "affectedClasses": classes,
  "affectedPods": pods,
  "httpStatuses": status_counts,
  "firstErrorAt": first_error,
  "lastErrorAt": last_error,
}
```

4.5 LLM use inside LogsAgent

If an LLM is used at all, feed it structured patterns or chunk summaries rather than the entire raw telemetry stream. The prompt must explicitly prohibit root-cause determination and remediation recommendations.

Preferred rule

Raw logs -> deterministic normalization/pattern extraction -> optional structured summary. This keeps the stronger platform principle that the model reasons over normalized knowledge rather than uncontrolled telemetry.

4.6 Retry behavior

Condition	Result
ELF timeout / network / transient 5xx / 429	Technical retry with bounded backoff
Malformed response	Fail LogsAgent and record error
Zero log results	Successful evidence result
No ERROR events	Successful healthy/negative evidence

4.7 Example evidence

```
{
  "agent_name": "LogsAgent",
  "evidence_type": "LOG",
  "source_system": "ELF",
  "title": "NullPointerException observed in application logs",
  "summary": "482 NullPointerException events observed.",
  "confidence": 0.96,
  "findings": {
    "errorCount": 482,
    "affectedClasses": ["PaymentProcessor"],
    "firstErrorAt": "2026-08-05T10:12:15Z"
  }
}
```

5. MetricsAgent - Runtime Health Evidence

MetricsAgent should be deterministic. It collects CPU, memory, JVM, HTTP, database and response-time signals and converts time series into factual statistics, health classifications and baseline comparisons.

5.1 Flow

```

MetricEvidenceRequest
  |
  v
MetricsQueryBuilder
  |
  v
MetricsTool -> Prometheus / Datadog / Dynatrace
  |
  v
MetricNormalizer
  |
  v
MetricAnalyzer
  |
  v
MetricHealthEvaluator
  |
  v
MetricEvidenceBuilder
  |
  v
Evidence[]

```

5.2 Request model

```

class MetricEvidenceRequest(BaseModel):
    investigation_id: UUID
    service_name: str
    environment: str
    start_time: datetime
    end_time: datetime
    required_metrics: list[str] = []
    namespace: str | None = None
    cluster: str | None = None
    pod: str | None = None
    step_seconds: int = Field(default=60, ge=15, le=300)

```

5.3 Normalized metric model

```

class MetricSample(BaseModel):
    timestamp: datetime
    value: float

class NormalizedMetricSeries(BaseModel):
    metric_name: str
    service: str
    environment: str
    unit: str | None = None
    labels: dict[str, str] = {}
    samples: list[MetricSample]

```

5.4 Deterministic analysis

For each series calculate sample count, minimum, maximum, average, first value, last value, delta and percent change. Preserve both service-level aggregation and instance/pod breakdown so one hot pod is not hidden by a healthy average.

5.5 Baseline comparison

A metric becomes much stronger evidence when compared with a baseline window. For example, CPU 67% is ambiguous; CPU baseline 28% -> current 67% is a meaningful increase.

Metric	Baseline	Incident Window	Interpretation
CPU	28%	31%	Healthy / stable

Metric	Baseline	Incident Window	Interpretation
Memory	60%	62%	Healthy / stable
HTTP 5XX	1.2%	18.6%	Unhealthy spike
P95 latency	240 ms	2.8 s	Unhealthy spike

5.6 Example evidence

```
{
  "agent_name": "MetricsAgent",
  "evidence_type": "METRIC",
  "source_system": "Prometheus",
  "summary": "HTTP 5XX and latency increased while CPU and memory stayed normal.",
  "confidence": 0.99,
  "findings": {
    "CPU": {"baselineAvg": 0.28, "currentAvg": 0.31, "health": "HEALTHY"},
    "HTTP_5XX_RATE": {"baselineAvg": 0.012, "currentAvg": 0.186, "health": "UNHEALTHY"}
  }
}
```

6. DeploymentAgent - GitHub and Change Evidence

DeploymentAgent is an MVP evidence specialist for deployment history, Git commits, releases, rollbacks and configuration changes. It must not interpret application logs or infer that a change caused the incident.

6.1 Flow

```
DeploymentEvidenceRequest
  |
  v
DeploymentTool
  |
  +--> GitHub deployments
  +--> GitHub commits
  +--> releases / workflow runs
  |
  v
DeploymentNormalizer
  |
  v
DeploymentAnalyzer
  |
  v
EvidenceBuilder
  |
  v
Evidence[]
```

6.2 Request model

```
class DeploymentEvidenceRequest(BaseModel):
    investigation_id: UUID
    service_name: str
    environment: str
    repository: str
    start_time: datetime
    end_time: datetime
    branch: str | None = None
    deployment_environment: str | None = None
    required_evidence: list[str] = []
```

6.3 Normalized event

```
class NormalizedDeploymentEvent(BaseModel):
    event_type: str
    timestamp: datetime
    repository: str
    environment: str | None = None
    version: str | None = None
    commit_sha: str | None = None
    branch: str | None = None
    actor: str | None = None
    workflow_name: str | None = None
    workflow_run_id: str | None = None
    status: str | None = None
    changed_files: list[str] = []
    rollback: bool = False
    reference_url: str | None = None
```

6.4 Deterministic analysis

The analyzer should identify the latest deployment before the incident, time delta from deployment to incident, commit count, included commits, release version, rollback presence and configuration file changes.

```
findings = {
    "deploymentDetected": latest is not None,
    "latestDeployment": latest_event,
    "minutesBeforeIncident": minutes_before_incident,
    "commitCount": len(commits),
    "commits": commit_summaries,
    "rollbackDetected": bool(rollbacks),
}
```

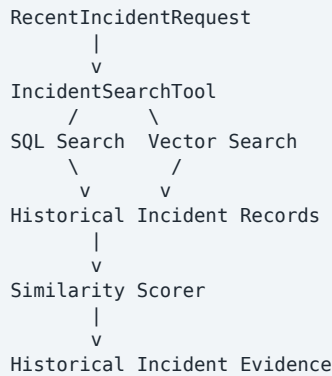
6.5 Example evidence

```
{
  "agent_name": "DeploymentAgent",
  "evidence_type": "DEPLOYMENT",
  "source_system": "GitHub",
  "title": "Deployment detected before incident",
  "summary": "Version 2.8.1 was deployed 18 minutes before the incident.",
  "confidence": 0.99,
  "findings": {
    "version": "2.8.1",
    "commitSha": "abc123",
    "minutesBeforeIncident": 18,
    "rollbackDetected": false
  }
}
```

7. RecentIncidentAgent - Historical Context

RecentIncidentAgent extends the evidence architecture with historical incident context. The simple implementation uses deterministic PostgreSQL filters; the enterprise evolution uses semantic similarity with pgvector.

7.1 Architecture



7.2 Request model

```

class RecentIncidentRequest(BaseModel):
    investigation_id: UUID
    service_name: str
    environment: str
    alert_type: str | None = None
    metric_name: str | None = None
    category: str | None = None
    lookback_days: int = 90
    top_k: int = 5
  
```

7.3 Deterministic similarity first

The first implementation can score stable metadata such as same service, environment, alert type, metric and category. The similarity score answers "how similar is the historical incident metadata?" - not "how likely is the same root cause?"

Match	Example Weight
Same service	0.40
Same environment	0.20
Same alert type	0.20
Same metric/category	0.20 combined

7.4 Enterprise semantic search



Historical root causes remain contextual evidence only. A previous incident that ended in configuration regression must never be copied into the current incident as a fact.

7.5 Example evidence

```
{
  "agent_name": "RecentIncidentAgent",
  "evidence_type": "HISTORICAL_INCIDENT",
  "source_system": "AIR_KNOWLEDGE_BASE",
  "title": "3 similar historical incidents found",
  "confidence": 0.88,
  "findings": {
    "matches": [
      {
        "incidentNumber": "INC-20260711-0012",
        "similarityScore": 0.90,
        "previousRootCause": "Configuration regression after deployment"
      }
    ]
  }
}
```

8. Evidence Aggregation and InvestigationKnowledge

Collected evidence should not go directly to the RCA model. AIR first converts a bag of observations into deterministic InvestigationKnowledge: ordered timelines, groups, health signals, relationships, contradictions and historical context.

8.1 Evidence Aggregator responsibilities

- Merge evidence from all completed agents.
- Deduplicate or link duplicate observations.
- Sort by time and build an incident timeline.
- Report missing or failed requested agents.
- Preserve source references for auditability.

8.2 AggregatedEvidence model

```
class AggregatedEvidence(BaseModel):
    investigation_id: str
    evidence: list[Evidence]
    missing_agents: list[str] = []
    duplicate_count: int = 0
    timeline: list[TimelineEvent] = []
```

8.3 Evidence Intelligence Engine

The Evidence Intelligence Engine should initially support deterministic relationship types such as TEMPORAL, HEALTH, CO_OCCURRENCE and HISTORICAL_SIMILARITY. It may say that exceptions started 18 minutes after deployment; it must not say that the deployment caused the exceptions.

```
Deployment 09:54
  |
  | +18 minutes
  v
Application exception 10:12
  |
  v
HTTP failure increase

Relationship = TEMPORAL
Root cause   = still unknown
```

8.4 InvestigationKnowledge model

```
class InvestigationKnowledge(BaseModel):
    investigation_id: str
    timeline: list[TimelineEvent]
    evidence_groups: list[EvidenceGroup]
    relationships: list[Relationship]
    healthy_components: list[str]
    affected_components: list[str]
    contradictions: list[Contradiction]
    historical_context: list[HistoricalContext] = []
    investigation_summary: InvestigationSummary
```

8.5 Example knowledge object

```
{
  "timeline": [
    {"time": "09:54Z", "type": "DEPLOYMENT", "event": "Version 2.8.1 deployed"},
    {"time": "10:12Z", "type": "APPLICATION", "event": "NullPointerException spike"}
  ],
  "relationships": [
    {
      "type": "TEMPORAL",
      "source": "Deployment",
      "target": "ApplicationException",
      "confidence": 0.91,
      "reason": "Exceptions started 18 minutes after deployment."
    }
  ],
  "healthyComponents": ["CPU", "Memory"],
  "affectedComponents": ["PaymentProcessor", "HTTP 5XX"]
}
```

9. Handoff to Root Cause Assessment

Only after InvestigationKnowledge is ready should Feature 4 invoke the RCA reasoner. The RCA Agent receives consolidated knowledge rather than raw telemetry and returns a structured RootCauseAssessment.

9.1 RCA output

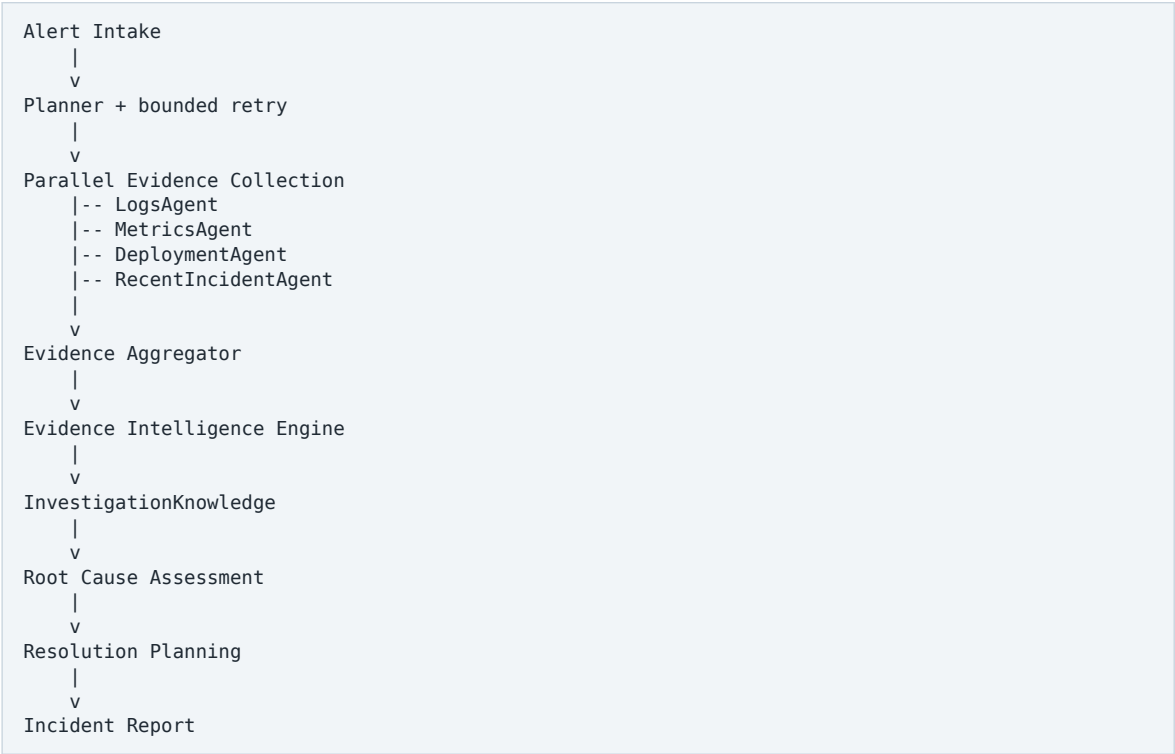
```
class RootCause(BaseModel):
    category: str
    title: str
    description: str
    confidence: float

class RootCauseAssessment(BaseModel):
    root_cause: RootCause
    supporting_evidence: list[str]
    alternative_causes: list[AlternativeCause]
    missing_evidence: list[str]
    reasoning_summary: str
```

Unknown is a valid result

If the evidence is insufficient, return an explicit UNKNOWN / low-confidence assessment and identify missing evidence rather than fabricating a root cause.

9.2 Full end-to-end flow



10. Engineering Rules to Preserve

Principle	Implementation Rule
Evidence before reasoning	Keep external observations as auditable evidence before LLM interpretation.
Thin LangGraph nodes	Nodes orchestrate service calls and state updates; they do not contain business logic.
Tool adapters	Agents depend on abstract tools. Vendor APIs, authentication, retries and rate limits stay behind a
Structured outputs	Planner and RCA outputs are validated Pydantic contracts, not free-form text.
Bounded loops	Every retry/re-plan path has an explicit max iteration or fallback.
Failure isolation	One failed evidence agent should not terminate the entire evidence collection phase.
Negative evidence matters	No errors or healthy CPU can be important contradictory evidence.
Historical context is not truth	A similar past incident informs the investigation but does not prove the same current root cause.

10.1 Suggested next implementation milestone

The next implementation step is to build the EvidenceAggregator and EvidenceIntelligenceEngine as deterministic services, wire all evidence agents into a parallel LangGraph subgraph, persist InvestigationKnowledge, and then implement the structured RCA Agent.

Appendix A. Suggested Repository Layout

```
air-agent-app/
|-- agents/
|   |-- planner_agent.py
|   |-- base_evidence_agent.py
|   |-- logs_agent.py
|   |-- metrics_agent.py
|   |-- deployment_agent.py
|   |-- recent_incident_agent.py
|-- services/
|   |-- context_builder.py
|   |-- prompt_builder.py
|   |-- llm_service.py
|   |-- evidence_aggregator.py
|   |-- evidence_intelligence.py
|-- tools/
|   |-- elf_log_adapter.py
|   |-- prometheus_adapter.py
|   |-- github_adapter.py
|   |-- incident_search_tool.py
|-- models/
|   |-- planner_output.py
|   |-- investigation_state.py
|   |-- evidence.py
|   |-- investigation_knowledge.py
|-- repositories/
|-- graph/
|   |-- nodes/
|   |-- investigation_graph.py
|-- prompts/
|   |-- planner.md
|   |-- rca.md
|-- tests/
```

Appendix B. Testing Checklist

- Test: Planner READY path
- Test: Planner low-confidence retry
- Test: Planner retry exhausted / human review
- Test: LogsAgent normal logs
- Test: LogsAgent zero results
- Test: LogsAgent transient tool failure
- Test: MetricsAgent baseline comparison
- Test: MetricsAgent one-hot-pod case
- Test: DeploymentAgent deployment-before-incident case
- Test: DeploymentAgent no deployment case
- Test: RecentIncidentAgent no matches
- Test: RecentIncidentAgent high-similarity matches
- Test: Parallel evidence failure isolation
- Test: Aggregator missing-agent detection
- Test: Temporal relationship rule
- Test: Healthy-vs-affected classification
- Test: RCA UNKNOWN when evidence is insufficient